

Understand SQL Window Functions & Frames comprehensively (Zero to Hero)

Content

1. What are Window Functions? Why Use Them?
 2. Core Concepts: The **OVER()** Clause
 - **PARTITION BY**: Dividing the Window
 - **ORDER BY**: Ordering within the Window
 - **Frames**: Defining the Subset (The Heart of Your Question)
 - What are Frames?
 - Frame Syntax (**ROWS** vs. **RANGE**)
 - Frame Boundaries
 - **The Default Frame (Crucial!)**
 - When and Why to Specify Frames
 3. Types of Window Functions (with Examples)
 - Ranking Functions (**ROW_NUMBER**, **RANK**, **DENSE_RANK**, **NTILE**)
 - Value Functions (**LAG**, **LEAD**, **FIRST_VALUE**, **LAST_VALUE**, **NTH_VALUE**)
 - Aggregate Functions as Window Functions (**SUM**, **AVG**, **COUNT**, **MIN**, **MAX**)
 - Running Totals (Cumulative Sum)
 - Moving Averages
 4. Putting It All Together: Guidelines & Use Cases
 5. Conclusion & Key Takeaways
-

Step 1 - What are Window Functions? Why Use Them?

Window functions perform calculations across a set of table rows that are somehow related to the *current* row. Unlike traditional aggregate functions (like `SUM()`, `AVG()` used with `GROUP BY`), window functions **do not collapse** the rows. Instead, they return a value **for each row** based on the "window" of related rows.

The "**Window**" is the set of rows the function considers. You define this window using the `OVER()` clause.

Why Use Them?

- **Complex Calculations Easily:** Calculate rankings, running totals, moving averages, percentages of totals, access data from preceding or succeeding rows, etc., without complex self-joins or subqueries.
- **Readability:** Often makes queries cleaner and easier to understand compared to alternative methods.
- **Performance:** Can be more efficient than self-joins for certain tasks.

Analogy: suppose you have a list of students' scores.

- A `GROUP BY` aggregate might give you the *average score per class*. It collapses all students in a class into one row showing the average.
- A window function could show you *each student's score* alongside the *average score of their class* or *their rank within the class*. It keeps all student rows.

Step 2 - Core Concepts: The `OVER()` Clause

The `OVER()` clause is what makes a function a window function. It specifies how to partition and order the data for the function's calculation. It can contain three optional sub-clauses: `PARTITION BY`, `ORDER BY`, and the frame clause.

```
SELECT
    column1,
    column2,
    WINDOW_FUNCTION_NAME(arguments) OVER (
        [PARTITION BY partition_expression]
        [ORDER BY sort_expression [ASC | DESC]]
        [frame_clause] -- ROWS or RANGE
    ) AS window_function_result
FROM your_table;
```

2.1 PARTITION BY: Dividing the Window

It Divides the rows of your query into partitions (groups). The window function is then applied *independently* to each partition. If you omit **PARTITION BY**, the entire result set is treated as a single partition.

You can think of **PARTITION BY** like **GROUP BY**, but it *doesn't collapse* the rows. It just creates separate calculation contexts.

It is used When you want the calculation to restart for different groups (e.g., rank employees *within each department*, calculate cumulative sales *per region*).

Example: Let's use a sample **employees** table:

emp_id	emp_name	department	salary	hire_date
1	Alice	Sales	60000	2021-01-15
2	Bob	Sales	75000	2020-03-10
3	Charlie	HR	55000	2021-05-20
4	David	IT	80000	2019-11-01
5	Eve	IT	95000	2020-08-15
6	Frank	HR	62000	2022-02-28

Query: Calculate the average salary for each employee's department alongside their details.

```
SELECT Emp_name, department, salary,  
       AVG(salary) OVER (PARTITION BY department) AS avg_dept_salary  
FROM  
employees;
```

Result:

emp_name	department	salary	avg_dept_salary
Charlie	HR	55000	58500.00

Frank	HR	62000	58500.00
David	IT	80000	87500.00
Eve	IT	95000	87500.00
Alice	Sales	60000	67500.00
Bob	Sales	75000	67500.00

Notice how **AVG(salary)** is calculated separately for HR, IT, and Sales. All rows are returned.

2.2 ORDER BY: Ordering within the Window

It Specifies the logical order of rows *within each partition*. This is crucial for functions that depend on sequence (like ranking functions, **LAG**, **LEAD**) and determines the meaning of **CURRENT ROW** and boundaries in the frame clause.

This **ORDER BY** *within* the **OVER()** clause defines the window function's calculation order. It might be different from the final **ORDER BY** clause of the entire query, which determines the final display order.

When to Use:

- Essential for ranking functions (**ROW_NUMBER**, **RANK**, **DENSE_RANK**).
- Essential for sequence functions (**LAG**, **LEAD**).
- Needed for meaningful calculations like running totals or moving averages.
- When using **RANGE** frames or when the default frame behavior with **ORDER BY** is desired.

Example: Rank employees within each department based on salary (highest first).

```
SELECT emp_name, department, salary,
RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank
FROM employees;
```

Result:

emp_name	department	salary	dept_rank
Frank	HR	62000	1
Charlie	HR	55000	2
Eve	IT	95000	1
David	IT	80000	2
Bob	Sales	75000	1
Alice	Sales	60000	2

The **ORDER BY salary DESC** inside **OVER()** determines the ranking order *within* each department partition.

2.3 Frames

A frame defines the **specific group of neighboring rows** used for the calculation at each step, relative to the current row. It lets you control the "window size" for functions like running totals or moving averages.

Why Needed? Frames allow you to perform calculations on a specific range of rows around the current row, like:

- Calculating a moving average (e.g., average of the current row and the previous 2 rows).
- Calculating a running total (e.g., sum from the start of the partition up to the current row).
- Comparing the current row to the very first or last row of the partition.

Frame Syntax: Specified within the **OVER()** clause after **ORDER BY** (if present).

```
OVER (  
  [PARTITION BY ...]  
  [ORDER BY ...]  
  frame_unit BETWEEN frame_start AND frame_end  
)
```

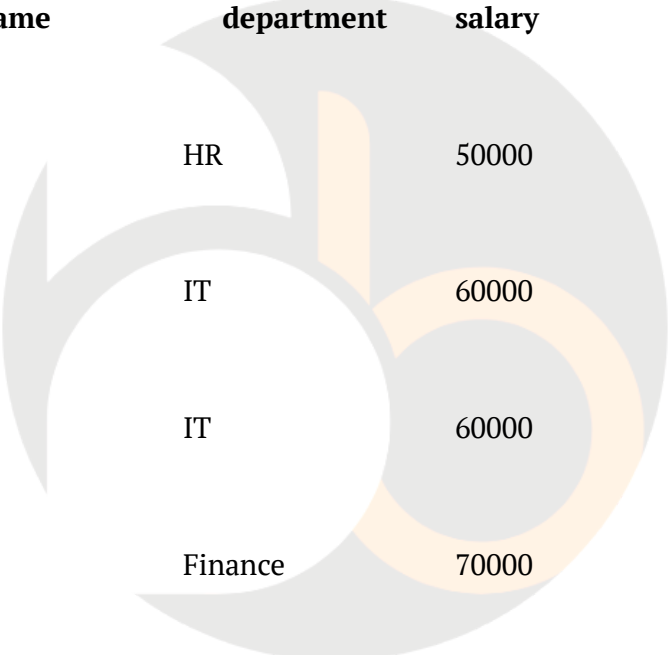
```
-- frame_unit: ROWS | RANGE  
-- frame_start/frame_end:
```

```
-- UNBOUNDED PRECEDING
-- n PRECEDING
-- CURRENT ROW
-- n FOLLOWING
-- UNBOUNDED FOLLOWING
```

ROWS vs RANGE Explained

Simple Example: Employee Salaries

Let's use a simple employee salary table: Notice that Bob and Charlie have the exact same salary.



employee_name	department	salary
Alice	HR	50000
Bob	IT	60000
Charlie	IT	60000
David	Finance	70000
Eve	HR	55000

What is ROWS?

ROWS counts physical rows - it treats each row as separate, even if they have the same values.

What is RANGE?

RANGE groups rows with the same values in your ORDER BY column - it treats rows with identical values as one logical group.

Let's See the Difference

I'll show a simple running total of salaries:

```
SELECT
    employee_name,
    salary,
    -- Running total using ROWS - treats each row individually
    SUM(salary) OVER (
        ORDER BY salary
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) as rows_total,

    -- Running total using RANGE - groups same values together
    SUM(salary) OVER (
        ORDER BY salary
        RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) as range_total
FROM
    employees;
```

Here's what we get:

employee_name	salary	rows_total	range_total
Alice	50000	50000	50000
Eve	55000	105000	105000

Bob	60000	165000	225000
Charlie	60000	225000	225000
David	70000	295000	295000

Look at the row for Bob:

- **rows_total = 165000**: This is 50000 + 55000 + 60000 (just Bob's salary)
- **range_total = 225000**: This is 50000 + 55000 + 60000 + 60000 (both Bob AND Charlie's salaries)

Then for Charlie:

- **rows_total = 225000**: Now it includes Charlie's salary too
- **range_total = 225000**: Same as before because RANGE already included all 60000 salaries

When to Use Each One

Use **ROWS** when:

- You want to count exactly N physical rows
- Each row should be treated separately, even if values are identical
- Example: "Give me the average of exactly the last 3 transactions"

Use **RANGE** when:

- You want to group identical values together
- You're thinking about value ranges rather than physical rows
- Example: "Give me the average of all transactions with the same price"

Real-World Example

In weather data analysis:

- **ROWS**: "Average temperature over the last 7 days exactly"
- **RANGE**: "Average temperature for all days between 60-70 degrees"

IMPORTANT NOTE:

Many SQL engines default to a full-range (**RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**) when no frame clause is explicitly provided.

Using ORDER BY:

When you add an ORDER BY, some databases change the default to **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** for aggregates, which means the window ends at the current row's value.

When to Use RANGE:

Use **RANGE** when your analysis depends on the relative value of the ordering column (for handling duplicates or dynamic group ranges) rather than a fixed number of rows. In contrast, use **ROWS** when you need a window defined strictly by row count.

Frame Boundaries:

- **UNBOUNDED PRECEDING**: The frame starts at the first row of the partition.
- **n PRECEDING**: The frame starts **n** rows (for **ROWS**) or rows with values **n** less (for **RANGE**) before the current row.
- **CURRENT ROW**: The frame starts or ends at the current row. For **RANGE**, this includes all rows with the same **ORDER BY** value as the current row.
- **n FOLLOWING**: The frame ends **n** rows (for **ROWS**) or rows with values **n** greater (for **RANGE**) after the current row.
- **UNBOUNDED FOLLOWING**: The frame ends at the last row of the partition.

2.4 The Default Frame (VERY IMPORTANT!)

The SQL standard defines default frames, which can sometimes surprise users:

If **ORDER BY** is *NOT* specified in **OVER()**:

- The default frame is **RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**.
- This means the frame **is the entire partition**.
- Example: **AVG(salary) OVER (PARTITION BY department)** calculates the average over *all* rows in the department because the default frame covers the whole partition.

If **ORDER BY** *IS* specified in **OVER()**:

- The default frame changes to **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**.
- **This is the key point of confusion!** Let's break it down:
 - **RANGE**: It considers values.
 - **UNBOUNDED PRECEDING**: Starts from the beginning of the partition.
 - **CURRENT ROW**: Ends at the current row *including all peers* (rows with the same **ORDER BY** value).
- **Implication for Aggregates**: When you use **SUM()**, **AVG()**, etc., with an **ORDER BY** in the **OVER()** clause but *without* explicitly specifying a frame, you often get a **cumulative** or **running** calculation by default.

Example: Default Frame

Query 1: Average salary per department (no **ORDER BY** in **OVER**)

```
SELECT emp_name, department, salary,  
       AVG(salary) OVER (PARTITION BY department) as avg_sal_no_order  
FROM employees;  
-- Result: avg_sal_no_order is the average of the ENTIRE department (Default:  
Whole Partition)
```

Query 2: Sum of salaries per department, ordered by salary (with **ORDER BY** in **OVER**)

```
SELECT emp_name, department, salary,
```

```

SUM(salary) OVER (PARTITION BY department ORDER BY salary) as
running_total_sal
FROM employees
ORDER BY department, salary; -- Final display order

```

Result (Conceptual - showing how *running_total_sal* is calculated):

emp_name	department	salary	running_total_sal	Frame (Default: RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
Charlie	HR	55000	55000	Sum(55000)
Frank	HR	62000	117000	Sum(55000, 62000)
David	IT	80000	80000	Sum(80000)
Eve	IT	95000	175000	Sum(80000, 95000)
Alice	Sales	60000	60000	Sum(60000)
Bob	Sales	75000	135000	Sum(60000, 75000)

Because **ORDER BY salary** is present, the default frame (**RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**) makes **SUM()** calculate a running total based on salary order within the department.

2.5 When and Why to Specify Frames Explicitly

You *must* specify a frame clause (**ROWS** or **RANGE BETWEEN ...**) when:

You need a frame *different* from the default.

- **Moving Averages/Sums:** You want to calculate over a fixed number of preceding/following rows (e.g., 3-month moving average). The default won't work. You need **ROWS BETWEEN 2 PRECEDING AND CURRENT ROW**.
- **Comparing to Partition Extremes (with **ORDER BY**):** If you use **ORDER BY** and want **FIRST_VALUE** or **LAST_VALUE** to consider the *entire* partition, not just up to the **CURRENT ROW**. You might need **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**. Be careful with **LAST_VALUE** and the default frame!
- **Calculations Centered Around Current Row:** E.g., average salary of the current employee, the one before, and the one after: **ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING**.

To improve clarity and avoid relying on default behavior. Even if the default frame gives the result you want (like a running total), explicitly writing **ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** can make the query's intent clearer to others (and your future self).

Example: Moving Average

Calculate a 3-period moving average of salary within each department, based on hire date.

```
SELECT emp_name, department, hire_date, salary,
       AVG(salary) OVER (PARTITION BY department ORDER BY hire_date
                        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW -- Explicit Frame!
                        ) AS moving_avg_salary_3period
FROM employees
ORDER BY department, hire_date;
```

Explanation:

- **PARTITION BY department:** Calculate independently for each department.
- **ORDER BY hire_date:** Define the sequence for the moving average.
- **ROWS BETWEEN 2 PRECEDING AND CURRENT ROW:** The frame includes the current row and the two rows immediately preceding it *based on the hire_date order within the department*. If there are fewer than 2 preceding rows (near the start of a partition), it uses however many are available.

Step 3 - Types of Window Functions (with Examples)

Let's use our `employees` table again.

3.1 Ranking Functions

These assign a rank based on the `ORDER BY` clause within each partition. `ORDER BY` is mandatory here.

ROW_NUMBER(): Assigns a unique, sequential integer to each row within its partition. Ties in `ORDER BY` get arbitrary but distinct numbers.

Use Case: Assigning a unique rank even if values are the same. Getting the "Top N" rows per partition reliably.

```
SELECT emp_name, department, salary,
       ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn
FROM employees;
-- Result: 1, 2 for HR; 1, 2 for IT; 1, 2 for Sales (Unique numbers)
```

RANK(): Assigns ranks based on the `ORDER BY` value. Rows with the same value get the same rank. Creates gaps in the sequence for subsequent ranks (e.g., 1, 1, 3).

Use Case: Standard competition ranking where ties share a rank, affecting the next rank.

```
SELECT emp_name, department, salary,
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk
FROM employees; -- Assuming Grace is added
-- Result for Sales: Bob (1), Grace (1), Alice (3) <- Gap after ties
```

DENSE_RANK(): Similar to `RANK()`, assigns the same rank to ties, but does *not* create gaps in the sequence (e.g., 1, 1, 2).

Use Case: Ranking where you want consecutive ranks even with ties.

```
SELECT emp_name, department, salary,
       DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS drnk
FROM employees; -- Assuming Grace is added
-- Result for Sales: Bob (1), Grace (1), Alice (2) <- No gap
```

NTILE(n): Divides the rows within each partition into a specified number (n) of ranked groups (buckets or tiles) as equally sized as possible. Returns the bucket number (1 to n).

Use Case: Dividing data into quartiles (NTILE(4)), deciles (NTILE(10)), etc., for analysis.

```
SELECT emp_name, department, salary,
       NTILE(2) OVER (PARTITION BY department ORDER BY salary DESC) AS
salary_group -- Divide into 2 groups per dept
FROM employees;
-- Result: Higher salary gets 1, lower gets 2 within each dept (approximately)
```

3.2 Value Functions (Positional)

These functions access the value of an expression from a different row within the partition relative to the current row. **ORDER BY** is usually mandatory to define "previous," "next," "first," and "last."

LAG(expression [, offset [, default]]): Accesses the value of *expression* from a *previous* row in the partition based on the **ORDER BY** clause.

- **offset**: How many rows back (default is 1).
- **default**: Value to return if the offset goes beyond the partition start (default is NULL).
- **Use Case**: Comparing current value to previous value (e.g., year-over-year change, difference from previous salary).

```
SELECT emp_name, department, salary, hire_date,
       LAG(salary, 1, 0) OVER (PARTITION BY department ORDER BY hire_date) AS
previous_salary_in_dept
FROM employees;
-- Shows the salary of the previously hired person in the same department.
```

LEAD(expression [, offset [, default]]): Accesses the value of *expression* from a *following* row in the partition based on the **ORDER BY** clause. Parameters are the same as **LAG**.

Use Case: Comparing current value to the next value (e.g., anticipating a future state based on the next record).

```
SELECT emp_name, department, salary, hire_date,
       LEAD(salary, 1, 0) OVER (PARTITION BY department ORDER BY hire_date) AS
next_salary_in_dept
FROM employees;
-- Shows the salary of the next hired person in the same department.
```

FIRST_VALUE(expression): Returns the value of *expression* from the *first* row in the current window frame.

Use Case: Getting the value associated with the top-ranked row (e.g., highest salary in the department) alongside every row.

Remember the default frame behavior if ORDER BY is used! You might need an explicit frame **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING** if you want the *partition's* first value when ordering.

```
SELECT emp_name, department, salary,
       FIRST_VALUE(emp_name) OVER (PARTITION BY department ORDER BY salary
DESC) AS highest_paid_emp_in_dept
-- Might need explicit frame in some DBs or scenarios for true partition
first:
-- FIRST_VALUE(emp_name) OVER (PARTITION BY department ORDER BY salary
DESC ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
highest_paid_emp_in_dept
FROM employees;
-- Shows the name of the highest-paid employee in that department on each
employee's row.
```

LAST_VALUE(expression): Returns the value of *expression* from the *last* row in the current window frame.

Use Case: Getting the value associated with the bottom-ranked row.

CRITICAL WARNING: Due to the default frame (**RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**) when **ORDER BY** is used, **LAST_VALUE** often returns the value from the *current* row unless you specify an explicit frame that extends to the end of the partition, like **ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING** or **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**.

```
SELECT emp_name, department, salary,
```

```

-- This often gives the CURRENT row's value due to default frame:
-- LAST_VALUE(emp_name) OVER (PARTITION BY department ORDER BY salary
DESC) AS potentially_wrong_last_value

-- Correct way to get the last value in the ordered partition:
LAST_VALUE(emp_name) OVER (
    PARTITION BY department
    ORDER BY salary DESC
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING -- Explicit
frame needed!
) AS lowest_paid_emp_in_dept
FROM employees;

```

NTH_VALUE(expression, n): Returns the value of **expression** from the **nth** row of the window frame (where **n** is 1-based). Like **LAST_VALUE**, often requires an explicit frame (**ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**) to consider the whole partition when **ORDER BY** is present. Can be less efficient than using **ROW_NUMBER** in a subquery for finding the Nth item.

Use Case: Finding the 2nd highest salary, 3rd hired employee, etc.

```

SELECT emp_name, department, salary,

    NTH_VALUE(emp_name, 2) OVER (
        PARTITION BY department
        ORDER BY salary DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING -- Need frame
for full partition access
    ) AS second_highest_paid_emp
FROM employees;

```

3.3 Aggregate Functions as Window Functions

Standard aggregate functions (**SUM**, **AVG**, **COUNT**, **MIN**, **MAX**) can be used as window functions by adding the **OVER()** clause. They operate on the current window frame.

SUM() OVER(...) / **AVG() OVER(...)** / etc.

Use Case: Calculate aggregates related to the current row without collapsing rows. Examples: total sales for the region on every sales transaction row, average order value for a customer on every order row.

Running Totals (Cumulative Sum): A common use case achieved often by the *default* frame when **ORDER BY** is present, or explicitly.


```
-- Using default frame (works because ORDER BY triggers the right default)
SELECT emp_name, department, hire_date, salary,
       SUM(salary) OVER (PARTITION BY department ORDER BY hire_date) AS
cumulative_salary_by_hire
FROM employees;
```

```
-- Explicit frame for clarity (achieves the same as above)
SELECT emp_name, department, hire_date, salary,
       SUM(salary) OVER (
         PARTITION BY department
         ORDER BY hire_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS cumulative_salary_by_hire_explicit
FROM employees;
```

Moving Averages: Requires an explicit frame clause.

```
-- 3-period moving average of salary by hire date within department
SELECT emp_name, department, hire_date, salary,
       AVG(salary) OVER (
         PARTITION BY department
         ORDER BY hire_date
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW -- Explicit frame essential
       ) AS moving_avg_3period
FROM employees;
```

Step 4 - Putting It All Together: Guidelines & Use Cases

- **Need Ranking?** -> Use **ROW_NUMBER**, **RANK**, **DENSE_RANK**. Requires **ORDER BY** in **OVER()**. Use **PARTITION BY** if ranking needs to restart for groups.
- **Need to Divide into Buckets?** -> Use **NTILE(n)**. Requires **ORDER BY**. Use **PARTITION BY** for groups.
- **Need to Compare with Previous/Next Row?** -> Use **LAG** or **LEAD**. Requires **ORDER BY**. Use **PARTITION BY** for groups.
- **Need First/Last Value in Partition/Frame?** -> Use **FIRST_VALUE**, **LAST_VALUE**. Requires **ORDER BY**. Be very careful with **LAST_VALUE**'s default frame! Specify **ROWS BETWEEN UNBOUNDED**

PRECEDING AND UNBOUNDED FOLLOWING if you need the true partition last value when ordering. Use PARTITION BY for groups.

- **Need Nth Value?** -> NTH_VALUE. Requires ORDER BY. Needs careful frame specification like LAST_VALUE. Often, using ROW_NUMBER in a subquery and filtering WHERE rn = N is simpler/more efficient.
- **Need Aggregation Alongside Detail Rows?** -> Use SUM, AVG, COUNT, etc., OVER().
 - **Aggregate over the entire group?** -> Use PARTITION BY only. (Default frame is the whole partition).
 - **Running Total/Cumulative Aggregate?** -> Use PARTITION BY and ORDER BY. The default frame (RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) often works, but ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW is more explicit and safer if order duplicates exist.
 - **Moving Aggregate (e.g., X-period average)?** -> Use PARTITION BY, ORDER BY, and an explicit ROWS BETWEEN <start> AND <end> frame (e.g., ROWS BETWEEN 2 PRECEDING AND CURRENT ROW).
- **Deciding on Frames:**
 - **Default OK?** If you need the whole partition (no ORDER BY) or a standard running total (ORDER BY present), the default *might* be fine, but be aware of what it is.
 - **Need a Specific Range?** If you need a moving window (n PRECEDING/FOLLOWING) or need to guarantee the *entire* partition is considered with ORDER BY (especially for LAST_VALUE, NTH_VALUE), you **must** specify the frame explicitly using ROWS or RANGE. ROWS is often more predictable unless you specifically need RANGE's value-based grouping.

Step 5 - Conclusion & Key Takeaways

- Window functions are powerful tools for analytical queries in SQL.
- They perform calculations across related rows **without collapsing** them.
- The OVER() clause is key, defining the partition (PARTITION BY), order (ORDER BY), and frame (ROWS/RANGE BETWEEN ...).
- **Frames are crucial:** They define the precise subset of rows used by many window functions, especially aggregates.
- **Understand the Default Frame:** It changes based on the presence of ORDER BY in the OVER() clause. This is vital for aggregates (SUM, AVG...) and value functions like LAST_VALUE. When in doubt, specify the frame explicitly.
- Choose the right function (Ranking, Value, Aggregate) based on your specific analytical need.

If you are in hurry, then prefer this one

Understanding SQL Window Functions: Step-by-Step

Window functions are powerful SQL tools that perform calculations across a set of table rows related to the current row. Unlike **GROUP BY** aggregates, they **do not collapse rows**; they return a value for *every* row.

1. The Core: The **OVER()** Clause

This clause is what makes a function a window function. It tells the database *how* to group and order the data for the calculation. It has three optional parts:

- **PARTITION BY <columns>**
 - **Purpose:** Divides the rows into independent groups (partitions). The window function calculation restarts for each group.
 - **Think:** Like **GROUP BY**, but it *doesn't merge* the rows.
 - **Example:** **OVER (PARTITION BY department)** - calculations will be done separately for Sales, HR, IT, etc. If omitted, the whole dataset is one partition.
- **ORDER BY <columns> [ASC|DESC]**
 - **Purpose:** Sorts the rows *within* each partition. This order is crucial for many functions.
 - **Think:** Arranging the rows within each group before calculating.
 - **Importance:**
 - Required for ranking (**RANK()**, **ROW_NUMBER()**).
 - Required for accessing previous/next rows (**LAG()**, **LEAD()**).
 - **Crucially, adding ORDER BY changes the default calculation window (frame)!** (More on this below).
 - **Example:** **OVER (PARTITION BY department ORDER BY salary DESC)** - Within each department, sort employees from highest salary to lowest before calculating.
- **Frame Clause (ROWS or RANGE BETWEEN ...)**

- **Purpose:** Defines the specific *subset* of rows (the "frame" or "sliding window") within the partition to use for the calculation, relative to the current row.
 - **Think:** A smaller window sliding through the partition. We'll dive deep into this next.
-

2. Understanding Frames (The Sliding Window)

Frames give you fine-grained control over which rows are included in a window function's calculation (especially for aggregates like **SUM**, **AVG**).

- **Why Use an Explicit Frame?**

- To calculate **moving averages/sums** (e.g., "average of the last 3 rows").
- To get the **true LAST_VALUE** in a partition when using **ORDER BY**.
- To make your query's intent **crystal clear**.

- **Frame Units: ROWS vs. RANGE**

- **ROWS (Physical Count)**

- **What:** Defines the frame by a fixed number of physical rows before/after the current row.
- **CURRENT ROW means:** Just the single row being processed.
- **Example: ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING** - Includes the row physically before the current one, the current row itself, and the row physically after it (based on the **ORDER BY** within **OVER()**).
- **Intuition:** Good for "fixed number of periods" logic (like a 3-month moving average).

- **RANGE (Value-Based)**

- **What:** Defines the frame based on a range of *values* compared to the current row's value in the **ORDER BY** column. (Requires **ORDER BY**, usually with one column).
- **CURRENT ROW means:** The current row **plus all other rows with the exact same ORDER BY value** (these are called "peers"). This is a key difference!
- **Example: ORDER BY salary RANGE BETWEEN 1000 PRECEDING AND CURRENT ROW** - Includes the current row (and any peers with the same salary) plus all preceding rows whose salary is between **current_salary - 1000** and **current_salary**.
- **Intuition:** Good when value proximity matters (e.g., "sum all transactions within 5 minutes"). It's also how **default frames** work.

- **Frame Boundaries (Keywords used in BETWEEN ... AND ...)**

- **UNBOUNDED PRECEDING**: Start of the partition.
 - **n PRECEDING**: n rows before (**ROWS**) or rows with values up to n less (**RANGE**).
 - **CURRENT ROW**: The current row (plus peers for **RANGE**).
 - **n FOLLOWING**: n rows after (**ROWS**) or rows with values up to n more (**RANGE**).
 - **UNBOUNDED FOLLOWING**: End of the partition.
-

3. Default Frames (What SQL Does Automatically - VERY Important!)

If you don't explicitly write a **ROWS** or **RANGE** clause, SQL uses a default frame. This default **depends entirely on whether you used ORDER BY inside OVER()**.

- **Scenario 1: NO ORDER BY in OVER()**
 - **Default Frame**: **RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**
 - **Meaning**: The frame is the **entire partition**.
 - **Effect**: Aggregates (**SUM**, **AVG** etc.) calculate over the whole group (e.g., **AVG(salary) OVER (PARTITION BY department)** gives the total average department salary).
 - **Scenario 2: ORDER BY IS PRESENT in OVER()**
 - **Default Frame**: **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**
 - **Meaning**: The frame includes rows from the start of the partition up to the current row's **value**, *including all peers* (rows with the same **ORDER BY** value).
 - **Effect**:
 - Aggregates (**SUM**, **AVG** etc.) become **cumulative** by default (e.g., **SUM(sales) OVER (ORDER BY month)** gives a running total of sales).
 - **LAST_VALUE() PITFALL**: This default frame makes **LAST_VALUE()** return the value from the *current* row (or its peers), NOT the actual last value in the partition order. You need an explicit frame for that!
-

4. Window Function Types & Examples

Let's look at common functions:

(A) Ranking Functions (Assign ranks within the partition)

- **ROW_NUMBER()**
 - **Purpose:** Assigns unique sequential numbers (1, 2, 3, 4...).
 - **Requires ORDER BY?** Yes.
 - **Frame Sensitive?** No.
 - **Key Usage:** Getting unique ranks even with ties; selecting Top N.
 - **Example:** Assign rank 1, 2, 3... to employees by hire date.
- **RANK()**
 - **Purpose:** Assigns ranks; ties get the same rank; creates gaps (1, 2, 2, 4...).
 - **Requires ORDER BY?** Yes.
 - **Frame Sensitive?** No.
 - **Key Usage:** Competition-style ranking where ties share rank and affect subsequent ranks.
 - **Example:** Rank products by sales; two products tied for 2nd place mean the next rank is 4th.
- **DENSE_RANK()**
 - **Purpose:** Assigns ranks; ties get the same rank; NO gaps (1, 2, 2, 3...).
 - **Requires ORDER BY?** Yes.
 - **Frame Sensitive?** No.
 - **Key Usage:** Ranking where you want consecutive ranks despite ties.
 - **Example:** Rank employees by salary densely.
- **NTILE(n)**
 - **Purpose:** Divides rows into **n** roughly equal-sized groups (buckets).
 - **Requires ORDER BY?** Yes.
 - **Frame Sensitive?** No.
 - **Key Usage:** Creating quartiles (**NTILE(4)**), deciles (**NTILE(10)**), etc.
 - **Example:** Group customers into 4 tiers based on spending.

(B) Value Functions (Access values from other rows)

- **LAG(column [, offset [, default]])**
 - **Purpose:** Gets value from a *previous* row.
 - **Requires ORDER BY?** Yes.
 - **Frame Sensitive?** Low (logic relies on order offset, not frame boundaries).
 - **Key Usage:** Comparing current row value to the previous one (e.g., month-over-month change).
 - **Example:** `LAG(sales, 1, 0) OVER (ORDER BY month)` gets the previous month's sales.
- **LEAD(column [, offset [, default]])**
 - **Purpose:** Gets value from the following row.
 - **Requires ORDER BY?** Yes.
 - **Frame Sensitive?** Low.
 - **Key Usage:** Comparing current row value to the next one.
 - **Example:** `LEAD(status, 1) OVER (ORDER BY event_time)` gets the status of the next event.
- **FIRST_VALUE(column)**
 - **Purpose:** Gets value from the *first* row in the current frame.
 - **Requires ORDER BY?** Recommended (for meaningful "first").
 - **Frame Sensitive?** Medium (depends on frame start). Default frames are usually okay.
 - **Key Usage:** Finding the first item in a group (e.g., earliest hire date in department).
 - **Example:** `FIRST_VALUE(hire_date) OVER (PARTITION BY department ORDER BY hire_date)`
- **LAST_VALUE(column)**
 - **Purpose:** Gets value from the *last* row in the current frame.
 - **Requires ORDER BY?** Recommended (for meaningful "last").
 - **Frame Sensitive?** Very High!
 - **Key Usage/Gotcha:** Finding the last item. **WARNING:** Default frame with `ORDER BY` gives WRONG result. **Must use explicit frame** like `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` to get the *partition's* actual last value.
 - **Example (Corrected):** `LAST_VALUE(salary) OVER (PARTITION BY department ORDER BY salary ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)`

- **NTH_VALUE(column, n)**
 - **Purpose:** Gets value from the *Nth* row (1-based) in the frame.
 - **Requires ORDER BY?** Yes.
 - **Frame Sensitive? Very High!**
 - **Key Usage/Gotcha:** Finding the Nth item. Similar to **LAST_VALUE**, it often needs an explicit frame covering the whole partition when **ORDER BY** is used. Can be less efficient than **ROW_NUMBER()**.
 - **Example (Corrected):** **NTH_VALUE(salary, 2) OVER (ORDER BY salary DESC ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)** gets the 2nd highest salary.

(C) Aggregate Functions as Window Functions (**SUM**, **AVG**, **COUNT**, **MIN**, **MAX**)

- **Purpose:** Perform standard aggregations but over the window frame, returning the result on *each* row.
 - **Requires ORDER BY?** Optional. Changes behavior drastically (see Default Frames).
 - **Frame Sensitive? Very High!** Calculation depends entirely on the rows included in the frame (default or explicit).
 - **Key Usage:**
 - **Group Aggregates:** No **ORDER BY** -> Calculates over whole partition (e.g., **AVG(salary) OVER (PARTITION BY department)**).
 - **Running Totals/Averages (Cumulative):** With **ORDER BY** -> Default frame (**RANGE ... CURRENT ROW**) often does this automatically. Explicit frame (**ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**) is clearer.
 - *Example:* **SUM(sales) OVER (ORDER BY order_date)** for cumulative sales.
 - **Moving Averages/Sums:** With **ORDER BY** + **explicit ROWS frame**.
 - *Example:* **AVG(price) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)** for a 7-day moving average.
-

5. **ROWS** vs. **RANGE** - Quick Comparison

Feature	ROWS	RANGE
Basis	Physical row count	Value relative to ORDER BY column
CURRENT ROW	Single, current row	Current row + Peers (same ORDER BY value)
Ties (Peers)	Treated as distinct rows	Grouped together
Requires	ORDER BY for context	ORDER BY (usually 1 col, value-comparable)
Use Case	Fixed # periods (Moving Avg)	Value proximity, Cumulative (Default Frame)
Default?	No	Yes (used by SQL's default frames)

Conclusion

Window functions are incredibly useful. The key to mastering them lies in understanding:

1. How PARTITION BY and ORDER BY set the stage.
2. What a **frame** is (ROWS vs. RANGE).
3. How the **default frame changes** based on ORDER BY.
4. When you **must specify an explicit frame** (moving aggregates, true LAST_VALUE/NTH_VALUE with ORDER BY).

A NOTE:

- **Order of Operations (Conceptual):** SQL queries generally follow a conceptual order of operations, including `FROM/JOIN`, `WHERE`, `GROUP BY`, Aggregate Functions (without `OVER()`), **Window Functions (with `OVER()`)**, `HAVING`, `SELECT`, `DISTINCT`, `ORDER BY`, and `LIMIT/OFFSET/Workspace`.
- **Conflict Between Window Functions and Standard Aggregate Functions:** You generally cannot directly use **most window functions** (like `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, `LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`, etc.) in the same `SELECT` list as standard aggregate functions (like `SUM`, `AVG`, `COUNT`) *without* a `GROUP BY` clause.
- **Why the Conflict Occurs:** The core reason is the difference in the level of aggregation. Standard aggregate functions aim to produce single summary values for a group of rows or the entire table, while **window functions operate on sets of rows and return a value for each individual row.**
- **Correct Ways to Combine Aggregate and Window Functions:**
 - **Using Subqueries:** Perform the aggregate calculation in a subquery, then join or select from it in the outer query where the window function is applied.
 - **Using Common Table Expressions (CTEs):** Define CTEs to first calculate either the aggregate value or apply the window function, and then combine the results in a subsequent query.
- **When to Use Each Type of Function:**
 - **Standard Aggregate Functions (without `OVER()`):** Use for summarizing data into single values or grouped values (with `GROUP BY`).
 - **Window Functions (with `OVER()`):** Use for performing calculations across a set of related rows without collapsing them, allowing you to see both individual row details and the calculation result.
- **Key Takeaway:** The incompatibility between direct use of standard aggregates and **most window functions** stems from their different levels of output. To use them together effectively when both summary values and row-level calculations are needed, separate the operations using subqueries or CTEs.

Exception:

It's worth noting that if you use an aggregate function as a *window function* (i.e., with the `OVER()` clause), then it *can* be used in the same `SELECT` list as other window functions. In this case, the aggregate function will also return a value for each row based on the defined window. For example:

```
SELECT
    order_id,
    order_amount,
    SUM(order_amount) OVER () AS total_amount, -- SUM as a window function
    NTILE(4) OVER (ORDER BY order_amount DESC) AS amount_quartile
FROM
    orders;
```

In this example, `SUM(order_amount) OVER ()` calculates the total amount across all rows (the entire window) and displays this total on each row, alongside the `NTILE` result. The key here is that `SUM` is being used *as a window function* with the `OVER()` clause.

So, the restriction applies when you try to mix a standard aggregate function (without `OVER()`) with a window function (with `OVER()`) in the same `SELECT` list without a `GROUP BY`.

TOP COURSES TO LEARN SQL

BEST SQL & Data Analytics Courses *100 100 100 100*

1. [Google Data Analytics](#)
2. [IBM Data Analyst](#)
3. [SQL for Data Science – UC Davis](#)
4. [Databases and SQL for Data Science with Python – IBM](#)
5. [Introduction to Querying Databases](#)
6. [Data Analysis with SQL – Google](#)
7. [Advanced SQL for Data Scientists – LinkedIn Learning](#)
8. [Analyzing Big Data with SQL – Cloudera](#)

9. [Data Science: Querying with SQL – HarvardX](#)
10. [Database Design & Basic SQL in PostgreSQL – University of Michigan](#)
11. [PostgreSQL for Everybody Specialization – University of Michigan](#)
12. [NoSQL Systems – UC San Diego](#)
13. [Data Management & Visualization – Wesleyan University](#)

.

